SDK & INTEGRATION GUIDE

Inaya Custody SDK

Client-side cryptographic sovereignty for distributed applications.

@inaya-network/custody-sdk · v1.0.13-beta · BNB Chain Testnet · October 2026

SECTION 01

Overview & Installation

Encrypt and fragment a file in under 5 lines of code. Files are encrypted and split into two independent shards entirely in the caller's own runtime, before anything ever leaves the device.

> Built on @noble/hashes and @noble/ciphers rather than the browser's crypto.subtle — deliberate, since React Native has no native SubtleCrypto implementation, and noble's pure-JS primitives produce byte-identical PBKDF2 keys and AES-GCM ciphertext, keeping shards decryptable across web, Node.js, and mobile alike.

INSTALLATION

```
$ npm install @inaya-network/custody-sdk ethers
```

- Requirements: Node.js 18+, ethers v6 as a peer dependency.
- Two modes: browser (via InayaKernel.connectWallet()) or Node.js/server-side (pass a raw ethers.Wallet directly).
- Published on the public npm registry — both the `beta` and `latest` dist-tags currently point at the same 1.0.13-beta release, so a plain install with no tag suffix works.
- Reproducible builds: every published release ships with a CHECKSUMS.md (custody-sdk/CHECKSUMS.md) listing the expected SHA256 of each distributed file, so a security-conscious integrator can rebuild from source and verify the published tarball matches it byte for byte, rather than trusting npm alone.

SECTION 02

Step-by-Step: Upload a File

STEP 1 — DERIVE A VAULT KEY

```
import { InayaKernel } from '@inaya-network/custody-sdk';

const passkey = 'user_secure_administrative_passkey';
const salt = InayaKernel.generateSecureSalt(16);

const vaultKey = await InayaKernel.deriveVaultKey({
  passkey, salt, iterations: 100000, algo: 'HMAC-SHA256',
});
```

STEP 2 — ENCRYPT & SLICE

```
const { filename, shardAlpha, shardBeta } = await InayaKernel.disperseAndSlice({
  file, encryptionKey: vaultKey,
});
// Pin shardAlpha/shardBeta to IPFS yourself (e.g. Pinata) — the SDK does not pin for you.
```

STEP 3 — APPROVE FEE TOKENS

```
const connection = await InayaKernel.connectWallet();
const { usdtFee, inayaFee } = await InayaKernel.approveFeeTokens({
  connection, fileSizeBytes: file.size,
});
```

STEP 4 — ANCHOR TO THE LEDGER

```
const result = await InayaKernel.anchorToLedger({
  connection, fileName: file.name, fileSizeBytes: file.size,
  dataShardAlpha: shardAlpha, dataShardBeta: shardBeta,
  onProgress: (e) => console.log(e.stage),
});
```

> Write-once by design: batchRegisterAssets() is the only function that writes asset data on-chain. Renaming/moving/deleting is handled entirely off-chain by the Metadata client, not by a contract call.

STEP 5 — RETRIEVE & RECONSTRUCT

```
const { name, owner, dataUrl } = await InayaKernel.retrieveAndReconstruct({
  connection, assetId: result.assetId, passkey,
  onProgress: (e) => console.log(e.stage),
});
```

SECTION 03

Staking

```
await InayaKernel.Staking.stake({ connection, amount, lockPeriodDays: 30 }); // 0, 30, or 90
await InayaKernel.Staking.unstake({ connection, amount });
await InayaKernel.Staking.claimReward({ connection });
```

> withdraw() and claimReward() are two separate on-chain actions — there is no combined "unstake and pay out" call.

SECTION 04

Payments — No-Wallet Card Flow

For customers who'd rather pay by card, InayaKernel.Payments is a typed client for your own backend's card-checkout routes — it contains zero secrets (no Stripe key, no treasury key), only calling fetch() against your own /api/* routes.

FLOW	BEHAVIOR
Corporate Reserve	Enterprise annual subscription, billed in USDT, maintenance in INAYA.
PAYG Checkout	Retail metered billing per upload.
Egress Checkout	Pay-per-retrieval unlock for a single file.

SECTION 05

Off-Chain Metadata Layer

Since Custody's on-chain write is permanent, InayaKernel.Metadata fills the gap for everything the contract can't do: renaming, moving, deleting, restoring, virtual folders, and sharing. Every mutating call is authenticated by a wallet signature, cross-checked against the real on-chain owner.

```
await InayaKernel.Metadata.renameFile({ connection, fileHash, newName });
await InayaKernel.Metadata.shareFile({ connection, fileHash, withAddress });
```

SECTION 06

Errors, Progress & Retries

- InayaValidationError — bad input caught before any network call.
- InayaWalletError — no provider, user rejected a signature.
- InayaContractError — an on-chain call reverted.
- InayaNetworkError — an RPC, IPFS gateway, or backend fetch failed.
- InayaError — shared base class for instanceof checks.

> Long-running calls accept onProgress and also emit matching events on InayaKernel.events. Transient failures (IPFS timeouts, read-only RPC calls) retry automatically with backoff; contract reverts and wallet rejections never do.

SECTION 07

Node Operators — @inaya-network/node-daemon

New since this guide was last written: a separate, published npm package for the operator side.

INSTALLATION — PUBLISHED TO NPM

```
$ npm install -g @inaya-network/node-daemon
```

- login — encrypts a wallet key at rest locally (PBKDF2 + AES-GCM).
- register <capacityGB> — registers on InayaNodeRegistry on-chain, plus off-chain capacity bookkeeping.
- start — a 5-minute heartbeat loop reporting telemetry.
- status — shows what the coordinator has recorded about this node: heartbeat recency, uptime score, daemon version/restarts, and threat-reporting reputation.
- report <indicator> — submits a signed threat observation to the Security Layer.
- service install / service uninstall — installs or removes the daemon as a native background service (Windows Service / systemd / launchd).

> It does not store or serve shards, and does not execute settlement/payout logic — an identity + registration + heartbeat agent today, not a full storage-node runtime.

SECTION 08

API Quick Reference

FUNCTION	PURPOSE
generateSecureSalt(bytes=16)	Cryptographically random salt.
deriveVaultKey(params)	PBKDF2-derives a 256-bit AES key for reuse.
disperseAndSlice(params)	AES-GCM-256 encrypt + midpoint-bisect into two shards.
reconstructAndDecrypt(params)	Rejoin shards, re-derive key, decrypt.
connectWallet()	Browser-only EIP-1193 wallet connect.
approveFeeTokens(params)	Reads live per-GB fees, approves spend.
anchorToLedger(params)	Registers shard CIDs on-chain (write-once).
retrieveAndReconstruct(params)	Reads on-chain CIDs, fetches shards, decrypts.
Staking.{stake,unstake,claimReward}	InayaStaking wrapper, 0/30/90-day lock tiers.
Payments.*	No-wallet card checkout client.
Metadata.*	Off-chain, signature-authenticated file mutations.
events / errors.*	Shared event emitter / typed error classes.

FUNCTION	PURPOSE
createPasskeyBackup / restorePasskeyBackup	Export/restore a passkey-derived vault as an encrypted, portable backup file.
TrustPlatform.{verifyAudit,getEvidence,checkPermission}	Client for Inaya's public trust/evidence API — verify an org's own hash-linked evidence chain, pull a record's evidence trail, check a capability gate.
IdentityIntegration.{signWebhook,sendEvent}	Client for Inaya's Identity Integration webhook API — sign and send a directory lifecycle event (joiner/mover/leaver) from your own AD/Entra/HR connector.
ComplianceProofs.{proveAuthorizedAccess,verifyAuthorizedAccessProof}	Real Schnorr/Fiat-Shamir zero-knowledge proof that a caller controls a key authorized for a given {policy, action, timestamp} — without revealing the key.
Intent.{create,discoverRoutes,simulate,riskCheck,execute}	Intent-based multi-chain transfer pipeline in front of the bridge SDK — validates, prices, risk-checks and executes a cross-chain transfer as one auditable unit.
Attestation.{verify,evaluatePolicy}	Verifies device/environment attestation evidence against a policy — software-simulated TEE today, honestly labeled non-hardware-backed.

FUNCTION	PURPOSE
Analytics.getWalletStorageStats()	Real per-wallet storage usage/cost statistics.
Backup.{getBackupStatus,getBackupHealth,getRedundancyStatus,requestRecovery}	Shard redundancy and recovery status for a stored file, plus a client to request recovery.
AppStore.{submitListing,getListings,getMyListings}	Client for the Web3 App Store — list and browse third-party applications built on Inaya.
Pqc. {generateDeviceKeyPair,wrapContentKeyHybrid,unwrapContentKeyHybrid,capabilityInfo,isHybridEnvelope}	Hybrid post-quantum key establishment (classical X25519 combined with ML-KEM-768 via a domain-separated HKDF) — additive to classical cryptography, not a replacement of it. Algorithm-agile sharing envelopes (LEGACY_CLASSICAL / HYBRID_PQC / PQC_REQUIRED) so existing classical-only shares keep working.

Security note — passkeys and derived vault keys should never be transmitted off-device or logged. Store only the salt and resulting CIDs in application state.

Two more published packages, same monorepo. @inaya-network/bridge-sdk (cross-chain $INAYA transfer and staking) and @inaya-network/react (InayaUploader / InayaFileBrowser / InayaConnect components) are both real, separately versioned npm packages alongside custody-sdk — out of scope for this guide, which covers custody-sdk specifically; ask your Inaya contact for their own reference material. A developer provisioning compute on their own hardware (rather than storage) instead wants the real, official aws CLI against Inaya's AWS CLI-compatible endpoint, the terraform-provider-inaya compute resources, or a downloaded kubeconfig for their own kubectl — see the Ecosystem Dev-Deepdive document's Sovereign Infrastructure Fabric reference for the full file-level surface.